CASOS PRUEBA

1. Tabla de Casos de Prueba

Caso	Entrada (minutos, cliente, perdido)	Resultado Esperado	Tipo de Caso	Regla(s)	Justificación
CP01	(30, "normal", false)	$20.00	Normal	**R6**	Evaluado dentro del rango base de 16 a 60 minutos.
CP02	(90, "normal", false)	$35.00	Normal	**R7**	1.ª hora ($20) + 1horaextrainiciad$ 15) = $35.
CP03	(180, "normal", false)	$50.00	Normal	**R7**	1.ª hora ($20) + 2horasadicionales$ 30) = $50.
CP04	(14, "normal", false)	$0.00	Frontera	**R5**	Dentro del margen de tolerancia gratuita (1 a 15 min).

CP05	(15, "normal", false)	$0.00	Frontera	**R5**	Límite superior de tolerancia gratuita.
CP06	(16, "normal", false)	$20.00	Frontera	**R6**	Primer minuto del rango con tarifa inicial de $20.
CP07	(59, "normal", false)	$20.00	Frontera	**R6**	Penúltimo minuto de la tarifa base fija.
CP08	(60, "normal", false)	$20.00	Frontera	**R6**	Límite superior de la primera hora con tarifa inicial.
CP09	(61, "normal", false)	$35.00	Frontera	**R7**	Primer minuto de la primera hora adicional iniciada ($20 + $15).
CP10	(-1, "normal", false)	Excepción (ValueError / IllegalArgumentException)	Inválido	**R1, R10**	Minutos negativos no permitidos; el sistema debe lanzar excepción.

CP11	(30, "vip", false)	Excepción (ValueError / IllegalArgumentException)	Inválido	**R2, R11**	Tipo de cliente no reconocido; el sistema debe lanzar excepción.
CP12	(61, "frecuente", false)	$31.50	Interacción / Decimal	**R7, R8**	$35 con 10\% de descuento$ (35 × 0.90 = $31.50). Pruebas con flotantes.
CP13	(500, "frecuente", true)	$300.00	Interacción	**R3, R8**	Boleto perdido cobra $300 fijos y previene aplicar descuento.
CP14	(121, "frecuente", false)	$45.00	Interacción	**R7, R8**	$50 tarifa base con 2 horas e$ (50 × 0.90 = $45).
CP15	(1000, "normal", false)	$260.00	Adicional	**R7, R9**	Escenario de estancia prolongada ($20 + 16 hrs extra × $15 = $260).

2. Reglas Cubiertas

Regla	Descripción	Casos que la Cubren
R1	Minutos enteros mayor o igual a 0.	CP01 a CP09, CP12 a CP15
R2	Tipo de cliente debe ser "normal" o "frecuente".	CP01 a CP09, CP12 a CP15
R3	Boleto perdido cuesta $300.00 sin importar tiempo ni cliente.	CP13
R4	Si minutos son 0, el costo es $0.00.	CP04 (evalúa zona gratuita ≤ 15 min)
R5	De 1 a 15 minutos el costo es $0.00.	CP04, CP05
R6	De 16 a 60 minutos el costo es $20.00.	CP01, CP06, CP07, CP08

R7	>60 min cobra $20.00 iniciales + $15.00 por hora adicional iniciada.	CP02, CP03, CP09, CP12, CP14, CP15
R8	Cliente frecuente recibe 10% de descuento (excepto con boleto perdido).	CP12, CP13, CP14
R9	El costo nunca puede ser negativo.	CP01 a CP09, CP12 a CP15
R10	Minutos negativos lanzan error.	CP10
R11	Tipo de cliente no válido lanza error.	CP11

3. Evidencia de Ejecución

3.1. Ejecución en Python (`pytest -v`)

```
PS C:\Users\jimen\Desktop\Practica4> pytest -v
================================= test session starts =================================
platform win32 -- Python 3.12.10, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\jimen\AppData\Local\Programs\Python\Pyth
on312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jimen\Desktop\Practica4
collected 16 items

python/test_estacionamiento.py::test_limites_tarifa_normal[14-0.0] PASSED          [  6%]
python/test_estacionamiento.py::test_limites_tarifa_normal[15-0.0] PASSED          [ 12%]
python/test_estacionamiento.py::test_limites_tarifa_normal[16-20.0] PASSED         [ 18%]
python/test_estacionamiento.py::test_limites_tarifa_normal[59-20.0] PASSED         [ 25%]
python/test_estacionamiento.py::test_limites_tarifa_normal[60-20.0] PASSED         [ 31%]
python/test_estacionamiento.py::test_limites_tarifa_normal[61-35.0] PASSED         [ 37%]
python/test_estacionamiento.py::test_30_minutos_cuestan_20 PASSED                  [ 43%]
python/test_estacionamiento.py::test_90_minutos_cuestan_35 PASSED                  [ 50%]
python/test_estacionamiento.py::test_180_minutos_cuestan_50 PASSED                 [ 56%]
python/test_estacionamiento.py::test_minutos_negativos_lanzan_error PASSED         [ 62%]
python/test_estacionamiento.py::test_cliente_invalido_lanza_error PASSED           [ 68%]
python/test_estacionamiento.py::test_61_minutos_cliente_frecuente_tiene_descuento PASSED [ 75%]
python/test_estacionamiento.py::test_boleto_perdido_cliente_cuesta_300 PASSED      [ 81%]
python/test_estacionamiento.py::test_121_minutos_cliente_frecuente_cuesta_45 PASSED [ 87%]
python/test_estacionamiento.py::test_1000_minutos_costo_no_negativo PASSED         [ 93%]
python/test_estacionamiento.py::test_costo_nunca_es_negativo PASSED                [100%]

================================== 16 passed in 0.71s ==================================
PS C:\Users\jimen\Desktop\Practica4>
```

3.2. Ejecución en Java (`mvn test`)

```
alejandrafloresleyva@MB-de-Alejandra src % mvn -q test
[ERROR] The goal you specified requires a project to execute but there is no POM in this directory (/Users/alejandrafloresleyva/Downloads/Practica4/Practica_4_Pruebas_Unitarias/java/src). Please verify
you invoked Maven from the correct directory. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MissingProjectException
alejandrafloresleyva@MB-de-Alejandra src % cd ..
alejandrafloresleyva@MB-de-Alejandra java % mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------< mx.edu:practica4-pruebas-unitarias >-----------------
[INFO] Building practica4-pruebas-unitarias 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] Loaded 24068 auto-discovered prefixes for remote repository central (prefixes-central.txt)
[INFO] Loaded 74 auto-discovered prefixes for remote repository apache.snapshots (prefixes-apache.snapshots.txt)
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ practica4-pruebas-unitarias ---
[INFO] skip non existing resourceDirectory /Users/alejandrafloresleyva/Downloads/Practica4/Practica_4_Pruebas_Unitarias/java/src/main/resources
[INFO]
[INFO] --- compiler:3.16.0:compile (default-compile) @ practica4-pruebas-unitarias ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ practica4-pruebas-unitarias ---
[INFO] skip non existing resourceDirectory /Users/alejandrafloresleyva/Downloads/Practica4/Practica_4_Pruebas_Unitarias/java/src/test/resources
[INFO]
[INFO] --- compiler:3.16.0:testCompile (default-testCompile) @ practica4-pruebas-unitarias ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.0:test (default-test) @ practica4-pruebas-unitarias ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running mx.edu.practica4.EstacionamientoTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.114 s -- in mx.edu.practica4.EstacionamientoTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.547 s
[INFO] Finished at: 2026-10-12T17:45:38-06:00
[INFO] ------------------------------------------------------------------------
```

4. Respuestas de Análisis

1. ¿Qué diferencia hay entre un caso normal y un caso frontera?

Un **caso normal** es un valor intermedio donde la regla se aplica sin estar en el límite de un cambio de tarifa. Por ejemplo, probar 30 minutos en tarifa normal es un caso normal: cae cómodamente en el rango de 16 a 60 minutos y sabemos que debe cobrar $20.00.

Un **caso frontera**, en cambio, evalúa los puntos exactos donde el sistema cambia de regla o de cobro. Por ejemplo, probar 15 y 16 minutos: en 15 todavía es gratis ($0.00), pero en 16 ya se cobra la tarifa base (20.00). También lo vemos entre 60 ($20.00) y 61 minutos ($35.00). Los casos frontera son donde más fácil se cometen errores al programar (como usar < en lugar de <=).

2. ¿Cuál de sus pruebas considera más importante y por qué?

Para nosotras , la prueba más importante es la del **boleto perdido con cliente frecuente (test_boleto_perdido_cliente_cuesta_300 / CP13)**.

Porque evalúa el cruce de dos reglas distintas (R3 y R8). Si el código no está bien ordenado, es muy fácil que aplique el 10% de descuento de cliente frecuente al total incluso cuando perdió el boleto (cobrando $270 en vez de $300). Esta prueba asegura que la penalización por boleto perdido tenga prioridad absoluta y no reciba ningún descuento.

3. ¿Encontró algún comportamiento de la implementación que no coincida con el modelo?

Sí, se encontró una discrepancia importante en la implementación inicial proporcionada en **Java** respecto a la regla **R7**.

En la versión original de JAVA el cálculo de las horas adicionales utilizaba la fórmula de división de enteros `(minutos - 60) / 60`. Esto provocaba que para `61` minutos el resultado fuera `0` horas adicionales en lugar de 1 hora extra iniciada, cobrando $20.00 en lugar de los $35.00 especificados por el modelo.

Por su parte, la versión en **Python** resolvía correctamente esto utilizando math.ceil ((minutos - 60) / 60).

Para solucionar esta incoherencia y hacer que el código respetara la regla R7, se corrigió el código Java ajustando la fórmula a (minutos - 60 + 59) / 60. De esta manera, el truncado de enteros en Java simula el redondeo hacia arriba (*ceiling*), permitiendo que la suite de pruebas en Java también apruebe al 100% todos los casos frontera.

4. ¿Una suite con 100% de pruebas aprobadas demuestra que el programa es correcto? Explique.

No. Que todas las pruebas pasen en la terminal solo significa que el programa funciona para las entradas específicas que decidimos probar en nuestra suite.

Si pasamos por alto probar valores extremos, combinaciones de tipos de cliente con boleto perdido o minutos en cero, el programa puede tener errores graves ocultos y aún así mostrar que pasó el 100% de los test.

5. Si una IA generará automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?

Creemos que deberíamos revisar lo siguiente:

1. **Que los valores esperados vengan del modelo y no del código:** Si la IA genera las pruebas leyendo el código existente, va a dar por "correcto" cualquier error que el programa contenga (como el fallo de división de enteros en Java).

2. **Que no sean 50 pruebas repetitivas:** Hay que verificar que realmente incluya casos frontera (15, 16, 60, 61 min), entradas inválidas (minutos negativos o clientes raros) y excepciones, y no solo variaciones del camino feliz.

3. **Manejo de flotantes:** Que en Java use tolerancias en los assertEquals y en Python use pytest.approx para evitar fallos por precisión decimal.